Authorisation and SSH

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

Authorisation and SSH: Lecture Overview

Authorisation at Systems Level: once identity is established, who decides what may be done, and on what basis

SSH and Its Limits: what public keys give you, and the two things they do not scale to

Certificates: a CA for users and hosts, principals as authorisation, and why expiry is not revocation

Hardening and the Hijack: the baseline directives, the capstone's open door, and agent forwarding

Into the Lab: the block's three answers to one question, and where Block 3 goes next

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

Identity Established, Access Decided

- Authentication answers **who**. Authorisation answers **what they may do**, and it is a separate decision
- Everything in this block so far was the first half: PAM, an identity provider, and today a certificate authority
- The second half is a policy question, and it is where the interesting failures are
- A system with perfect authentication and careless authorisation is a system with a well-identified intruder

Two questions, two mechanisms. Conflating them is how a correct login ends up with more rights than it should.

Four Models, Two Questions

- **DAC**, discretionary: the owner decides. Unix permissions. What you meet everywhere
- **MAC**, mandatory: a policy decides and the owner cannot override it. Block 3's AppArmor
- **RBAC**: rights attach to roles, and people hold roles. How organisations actually think
- **ABAC**: the decision is computed from attributes, including time, location and device state
- Two questions: **who sets the policy** (DAC, MAC), and **what a rule names** (roles, attributes)

Ask who sets the policy, then what a rule is written in. Most disagreements about access control mix the two.

The Unix Baseline Is Discretionary

- Owner, group, other, three bits each: the core of the model, on every file you will meet
- **Discretionary** means the owner may hand access out. Convenient, and it means policy is not centrally decidable
- Nobody can look at the system and say what the policy is: it is the sum of every choice every owner made
- That is the gap Block 3's MAC exists to close, and MAC binds `root` too, as DAC mostly does not
- Discretionary does not mean weak. It names **who decides**, and here the answer is each owner

Discretionary means the security policy is whatever the users have collectively done to it so far.

sudo Is a Policy File

- `sudoers` is the closest thing a stock Unix has to a written authorisation policy: who may run what, as whom
- `ALL=(ALL) ALL` says: this user may become anybody and run anything. It is a role, written as a blank cheque
- A scoped rule names the command, and that is the difference between delegation and handing over the machine
- Week 6 has you find a rule that looks scoped and is not, because of what the named command can do

Read a sudoers rule as a sentence: this person, may run this, as that identity. Every vague part is an attack surface.

The Reference Monitor, Revisited

- Week 1's ideal: **complete mediation**, **tamper-resistance**, **verifiability**
- Unix DAC mediates file access, but `root`'s capabilities override it and the kernel is too large to verify
- Later blocks wrap it: MAC adds a policy owners cannot override, capabilities shrink what `root` means
- Each layer narrows what the layer under it leaves open

Nothing here achieves the ideal. Knowing which of the three properties a mechanism gives you is the useful skill.

Least Privilege as the Through-Line

- Every mechanism from here serves one sentence: hold only what you need, only while you need it
- Today: a certificate that names one principal and expires in an hour, instead of a key that works forever
- Block 3: a capability instead of `root`. Block 4: a few syscalls instead of the whole kernel
- The direction of travel is always the same, and so is the cost: finer policy is more policy to get wrong

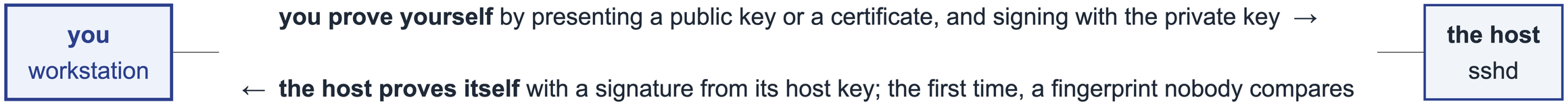
Least privilege is not a control. It is the question you ask of every control: could this be narrower?

SSH and Its Limits

Everybody knows how to add a public key to a server. What happens at the two hundredth server?

What SSH Actually Provides

- An **authenticated, confidential channel** to a remote host, and a way to authenticate you to it
- It is how most infrastructure is administered, which makes it the interface most worth attacking
- In this unit it is also capstone link 1: initial access to the target machine in December



Two directions: the host proves itself to you, and you prove yourself to the host. Both can fail, differently.

Public-Key Authentication

```
$ ssh-keygen -t ed25519          # once
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3Nza... you@laptop

# on the server, one line per key
$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3Nza... you@laptop
```

- The private key never leaves your machine. What crosses the wire is a **signature** tied to this session
- So the server learns you hold the key, and learns nothing that would let it impersonate you elsewhere
- This is Lecture 4's keypair idea, on a host instead of a website: same objects, smaller protocol
- And the whole of the trust relationship is that one line, copied by hand, on every server

Nothing secret is transmitted. The weakness is not the cryptography, it is the copying.

Limit 1: Distribution

- One line per user, per host. Fifty engineers and two hundred hosts is ten thousand lines that must agree
- Adding somebody means touching every host. **Removing** somebody means touching every host, and being sure
- There is no central revocation, and no way to answer which hosts does this key still open except by looking
- Configuration management distributes and audits the lines, but each host still enforces its own copy

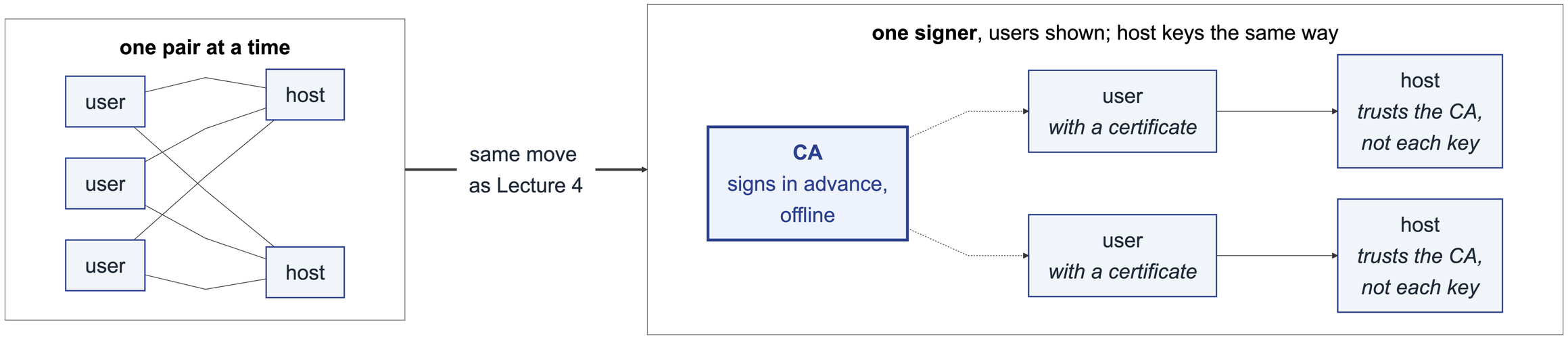
The failure mode is not a break-in. It is a key nobody removed, on a host nobody remembered.

Limit 2: Host Authenticity

- The other direction. How do you know the host you just reached is the one you meant?
- **Trust on first use**: the first connection shows a fingerprint nobody checks, and stores it in `known_hosts`
- The first connection proves the host holds a key, not that it is the right host's. It recurs on every rebuild
- Then `known_hosts` grows unpruned, so a real warning arrives among warnings people already ignore

Everyone types yes. The security of the first connection rests on a fingerprint almost nobody compares.

Two Gaps, One Answer



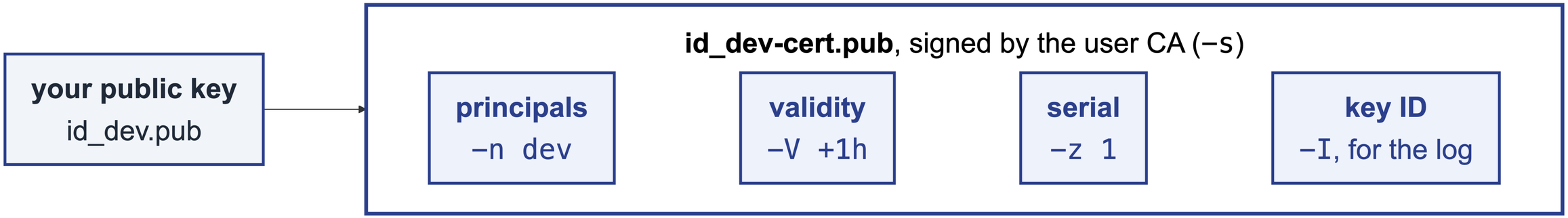
The same trust anchor as last week, one layer down. If it worked for websites, it works for hosts.

Certificates

A signature says who vouched for you. What decides which account that lets you become?

Signing a User Key

- `TrustedUserCAKeys` makes the host trust the **CA**, not the key. Ten thousand lines become one
- The certificate is written next to the key it signed, as `<name>-cert.pub`
- Sign two different keys for two certificates: signing one twice overwrites the first



Four flags, four fields: a name for the logs, the principals, the validity and the serial.

Principals Are Authorisation

- The CA vouches for the key and its **principals**. The target decides which **account** they may become
- With no `AuthorizedPrincipalsFile` a default rule decides. With one, the target's owner writes the policy
- Critical options bound the session: `force-command` runs one command, whatever was typed
- `source-address` narrows it further. Both travel **inside** the certificate, so the client cannot drop them



The CA vouches; local policy authorises.

Expiry Is Not Revocation

- A short validity **bounds exposure in time**. It does not end an issued certificate early
- To stop one **now**: a KRL that every host loads through `RevokedKeys`, or retire the old CA everywhere and reissue
- Four levers, four jobs. The common error is spending the third and believing you bought the fourth

authenticate
CA signature
vouches for the key and its principals

authorise
principals
what that buys here

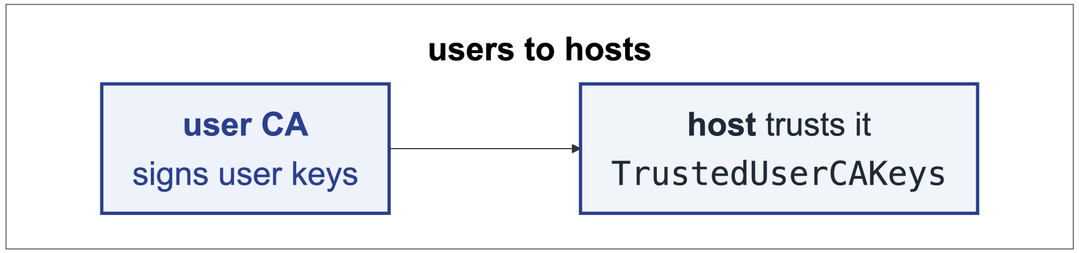
bound
validity interval
for how long

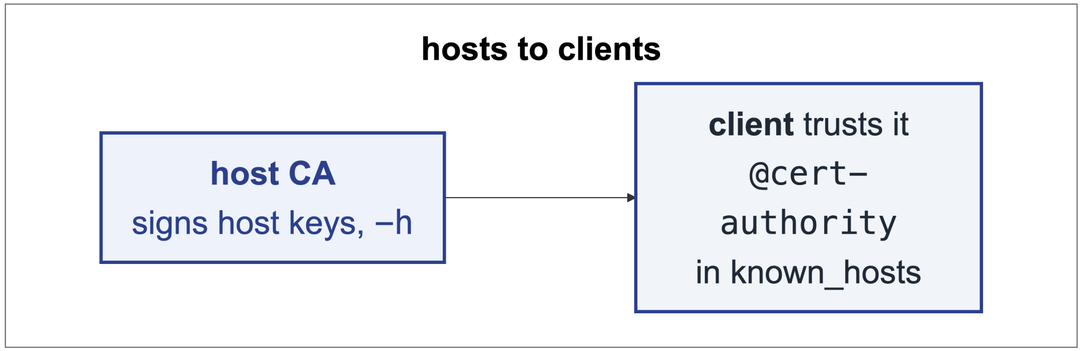
terminate
KRL, RevokedKeys
stop it early, on every host that loads the KRL

One hour of validity gives a thief up to one hour, and noticing the theft does not shorten it.

Host Certificates Close the Other Gap

- `-h` signs a **host** key rather than a user key. Everything else is the same ceremony
- One `@cert-authority` line replaces every `known_hosts` entry, for the domain's certified hosts
- Trust on first use disappears: the host presents a verifiable certificate, or it does not
- This closes Limit 2. In the laboratory it is an unmarked take-home extension





One line on the client, and the fingerprint prompt nobody reads is gone for the whole estate.

Hardening and the Hijack

The defaults let you in. Which of them would you turn off, and what does each one cost?

The sshd Baseline

```
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
RevokedKeys /etc/ssh/revoked_keys
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
AuthenticationMethods publickey
```

- `RevokedKeys` ends a certificate early: the threat is a leaked one you cannot wait out
- That is the pattern: name the directive, then the threat it closes. The other lines read the same way
- Choosing which of them matter most, and why, is the laboratory's work, not this slide's
- Each directive answers one threat. If you cannot say which, you are copying, not writing

Lab Core

This is the baseline the laboratory asks for. Every line is one sentence of policy, and you should be able to say the sentence.

The Capstone's Open Door

- The December target ships with `PasswordAuthentication` **on**. One default, deliberately left as it comes
- Where nothing else insists on a key, that line reopens the initial-access path. Saying why is the laboratory's job
- It is not a vulnerability in SSH. It is a decision nobody revisited, which is the shape of most real initial access
- Your job in December starts there, and this hour is where you learn what should have been done instead

Most intrusions do not start with an exploit. They start with something that was left on.

Agent Forwarding

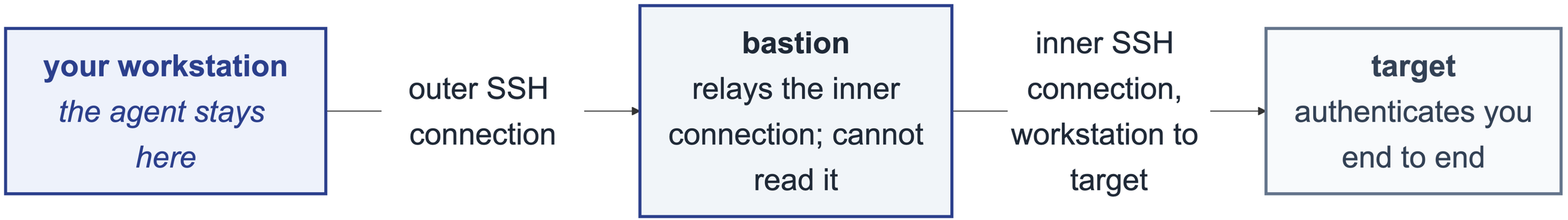
- `ssh -A` lets the machine you are on ask **your** agent to sign. Convenient across a jump host, and dangerous
- **Nothing is stolen.** Your private key never leaves your laptop; the socket on the bastion is enough to act as you
- Root on that bastion can use it for as long as your session lasts, against anything your key opens
- **An ordinary certificate does not save you**: it is a valid signature, made by your agent, on request
- And every login it makes carries your identity, which is the other half of the damage



An attack with no theft and no exploit. The credential stays home and is used anyway.

Use ProxyJump Instead

- `ssh -J bastion target` runs a second SSH connection, to the target, **inside** the one to the bastion
- The bastion relays a connection it cannot read, and no agent socket is left on it



One flag apart, and one of them hands a compromised host your identity for the length of the session.

Today in the Laboratory

Lab 5 closes lab report 2, with Labs 3 and 4. It is also capstone link 1: this is the door December opens.

- Stand up a user CA, issue two certificates, and drive an authorisation policy with principals
- Revoke one certificate, and find out what that does and does not change
- Harden `sshd` to the baseline you just read, on a target that starts without it
- Then run the agent-forwarding hijack across two containers, and defend it

Two identities, one CA, one revocation. Sign two separate keys: signing one twice leaves you one certificate.

Lab Core

Block 2 in One Slide

- Each step moves trust somewhere new, and the single point of failure moves with it

Week 3
local files
the machine
checks,
at login

Week 4
an identity provider
somebody else
decides,
at login

Week 5
a certificate
authority
somebody else
vouched,
in advance

Three answers to one question. The report asks you which you would deploy, and what you would do the day it fails.

Where Block 3 Goes

- So far: **who may act**. From next week: **what a privileged process can actually do**
- The Unix privilege model and set-UID, then capabilities, then mandatory access control
- The attacker stops being someone at the door and becomes a process already inside
- And the surface metric starts moving, one mechanism at a time

Authentication is finished. Everything after this is about limiting what an authenticated thing can do.

Reading and Reference, All Optional

- **ssh-keygen(1), sshd_config(5), ssh_config(5)**. The manual pages for everything on these slides. Read them on your own machine.
- **PROTOCOL.certkeys, in the OpenSSH source**. The certificate format: principals, critical options, extensions, in four pages. https://github.com/openssh/openssh-portable/blob/V_10_0_P1/PROTOCOL.certkeys
- **Facebook, Scalable and Secure Access with SSH, 2016**. Why an estate moves to certificates, by people who had to. https://engineering.fb.com/2016/09/12/security/scalable-and-secure-access-with-ssh/
- **Stallings and Brown, Computer Security: Principles and Practice, 4th ed., 2018**. Chapter 4 for the access-control models in the first section.
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. The set-UID chapter, which is where Week 6 starts.

Reference

Key Takeaways

- Authentication answers who; authorisation answers what. Two decisions, two mechanisms
- Unix access control is discretionary, so the policy is the sum of what everybody has done to it
- A user CA and a host CA replace ten thousand copied lines, one signer per direction
- **The signature vouches for the principals; local policy authorises them.** That separation is this hour's idea
- Expiry bounds exposure. Revocation ends it, where the KRL is loaded. Confusing the two is the classic error

And the hijack: an attack with no theft and no exploit. In the lab you run it, and then you fix it.